UNIVERSIDADE DO OESTE DE SANTA CATARINA
ENGENHARIA DE COMPUTAÇÃO
COMUNICAÇÃO DE DADOS E APLICAÇÕES DISTRIBUÍDAS

MONITORAMENTO REMOTO DA TEMPERATURA EM GRANJA AVÍCOLA

ISABELA CRISTINA MUNZLINGER
ISABELA FERNANDES
MARCOS EDUARDO RIBAS
VITOR BRUNO CHIAMULERA

JOAÇABA - SC
2026

1 INTRODUÇÃO

Este trabalho tem como tema o monitoramento remoto da temperatura em uma granja de frangos de corte. A proposta consiste em um sistema distribuído capaz de coletar a temperatura do ambiente, armazenar o histórico das leituras, apresentá-las em um painel de visualização e acionar automaticamente um mecanismo de resfriamento quando necessário.

1.1 Problema

Para o crescimento adequado dos pintinhos, é necessário manter a temperatura do aviário dentro de uma faixa ideal. Sendo assim, caso a temperatura ultrapasse o limite adequado, o sistema deve ser capaz de acionar uma ventoinha para reduzi-la.

1.2 Informação coletada e público-alvo

A informação coletada pelo sistema é a **temperatura** do ambiente da granja. O usuário final da solução é o **proprietário da granja**, que acompanha as leituras e o estado da ventoinha por meio do painel de visualização (dashboard).

2 JUSTIFICATIVA

As aves são animais homeotérmicos, ou seja, tendem a manter sua temperatura corporal constante, "nas aves domésticas em torno de 41 °C" (CARVALHO, 2010, p. 4). Entretanto, segundo Aviagen (2018), antes de completarem duas semanas de vida, os frangos não conseguem regular adequadamente sua temperatura corporal.

A temperatura do ambiente é um dos fatores que mais afetam a produtividade. Dessa forma, é imprescindível que ela se mantenha em uma faixa de conforto térmico, de modo a não gerar estresse por frio nem estresse por calor nas aves:

> O estresse causado pelo ambiente térmico influencia a produtividade dos animais por alterar sua troca de calor com o meio, modificando o consumo do alimento, o ganho de peso corporal e, consequentemente, a metabolizabilidade e as exigências por nutrientes (CARVALHO, 2010, p. 3).

Outro fator importante para a manutenção do conforto térmico, correlacionado com a temperatura, é a umidade relativa do ar (UR). Conforme Carvalho (2010, p. 6):

> A capacidade das aves em suportar o calor é inversamente proporcional ao teor de umidade relativa do ar. Quanto maior a umidade relativa do ar, mais dificuldade a ave tem de remover calor interno pelas vias aéreas, o que leva ao aumento da frequência respiratória.

Quanto à temperatura, Aviagen (2018) estabelece níveis considerados ideais para a criação de frangos de corte: 30 °C no primeiro dia de vida das aves, com redução gradual da temperatura até alcançar 20 °C aos 27 dias de idade.

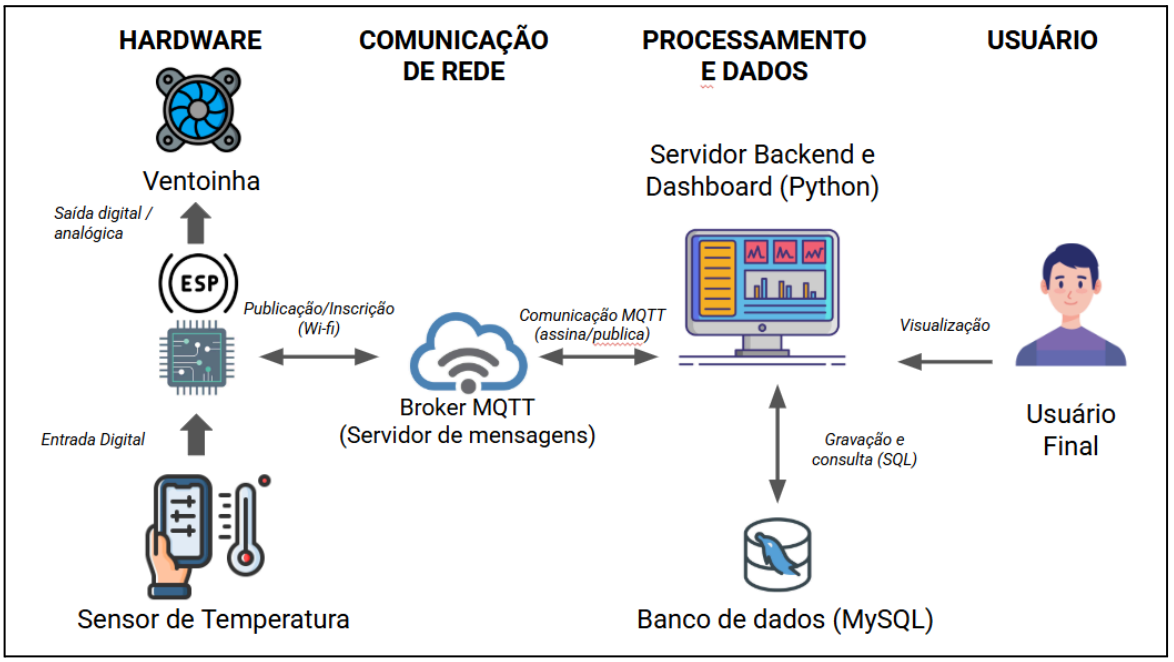
Por se tratar de um protótipo, adotou-se um limite fixo de 30 °C para o acionamento da ventoinha. Esse valor corresponde à maior temperatura recomendada por Aviagen (2018) ao longo de todo o ciclo de criação, de modo que temperaturas acima dele estão fora da faixa ideal em qualquer idade das aves. A adequação do limite à idade do lote, reduzindo-o gradualmente até 20 °C, fica como possível melhoria futura.

Ressalta-se ainda que, embora o sensor DHT22 também meça a umidade relativa do ar, esta versão do sistema utiliza apenas a temperatura no controle, ficando a umidade como possível ampliação. Da mesma forma, o protótipo atua somente no resfriamento do ambiente, por meio da ventoinha, e o controle de aquecimento não faz parte do escopo deste trabalho.

3 ARQUITETURA DO SISTEMA

A Figura 1 apresenta a visão geral da arquitetura proposta, organizada em quatro camadas: hardware, comunicação de rede, processamento e dados, e usuário.

Figura 1: Diagrama da arquitetura do sistema



Fonte: elaborado pelos autores (2026).

3.1 Componentes do sistema

O Quadro 1 descreve os componentes que integram o sistema e a função de cada um.

Quadro 1: Componentes do sistema

Componente	Tecnologia	Função
Dispositivo de processamento	ESP32	Lê o sensor, publica a temperatura e aciona a ventoinha
Sensor	DHT22	Mede a temperatura do ambiente (na versão atual, a leitura é simulada)
Saída (atuador)	Ventoinha	Reduz a temperatura do ambiente quando acionada
Intermediador de mensagens	Broker MQTT público (test.mosquitto.org)	Encaminha as mensagens entre o ESP32 e o backend
Backend	Script em Python	Executa a regra de negócio e a comunicação MQTT
Interface	Script em Python com Streamlit	Exibe o dashboard visual ao usuário
Banco de dados	MySQL	Armazena o histórico das leituras

Fonte: elaborado pelos autores (2026).

A comunicação entre o dispositivo e o backend ocorre por meio do protocolo MQTT, utilizando a rede Wi-Fi. O backend grava as leituras no banco de dados e o dashboard as consultas, ambos por meio de comandos SQL.

3.2 Papéis de cliente e servidor

O papel de cliente ou de servidor varia conforme a comunicação analisada. O ESP32 atua como cliente do Broker MQTT, publicando a temperatura e assinando o tópico de comando da ventoinha. O script do backend, em Python, também atua como cliente do Broker MQTT, para receber os dados e enviar comandos, e como cliente do banco de dados MySQL, para gravar as leituras. Já o script do dashboard atua como cliente do MySQL, para consultar o histórico, e como servidor web para o navegador do usuário. O Quadro 2 sintetiza esses papéis.

Quadro 2: Papéis de cliente e servidor em cada comunicação

Comunicação	Cliente	Servidor
ESP32 e Broker MQTT	ESP32	Broker MQTT
Backend e Broker MQTT	Backend (Python)	Broker MQTT
Backend e banco de dados	Backend (Python)	MySQL
Dashboard e banco de dados	Dashboard (Streamlit)	MySQL
Usuário e dashboard	Navegador do usuário	Dashboard (Streamlit)

Fonte: elaborado pelos autores (2026).

3.3 Modelo arquitetural

A arquitetura adotada é mais próxima do modelo cliente-servidor, com as responsabilidades organizadas da seguinte forma:

- **Interface:** dashboard;
- **Processamento:** backend em Python, responsável pela regra de negócio, com o ESP32 responsável pela leitura do sensor e pelo acionamento da ventoinha;
- **Dados:** MySQL.

Não se trata de uma arquitetura de microsserviços, pois esta exigiria maior granularidade, com a divisão do processamento em diferentes serviços independentes. Como a aplicação não possui uma demanda de escalabilidade que justifique essa separação, a coleta de temperatura e o controle da ventoinha podem ser realizados com uma arquitetura centralizada, organizada em camadas.

3.4 Fluxo de dados

O ciclo completo de funcionamento do sistema ocorre nas seguintes etapas:

1. **Origem:** o sensor DHT22 coleta a temperatura no ESP32. Na versão atual do protótipo, a leitura é simulada com valores aleatórios entre 25 °C e 35 °C, até a integração do sensor físico.
2. **Comunicação (envio):** o ESP32 publica a temperatura via MQTT, por meio da rede Wi-Fi, a cada 5 segundos.
3. **Recebimento (backend):** um script em Python, atuando como cliente MQTT, recebe o dado, salva no MySQL e toma a decisão com base na regra de negócio: acima de 30 °C, a ventoinha deve ser ligada; nos demais casos, desligada.
4. **Comunicação (retorno):** o script em Python publica no MQTT o comando de ligar (ON) ou desligar (OFF) a ventoinha.
5. **Ação:** o ESP32 recebe a mensagem do MQTT e aciona a ventoinha (relé ou LED no protótipo).
6. **Visualização:** o dashboard consulta o MySQL a cada 3 segundos e exibe a temperatura atual, o estado da ventoinha e o histórico das últimas 20 leituras.

Como melhoria futura, prevê-se a adoção de uma histerese de 1 °C, ou seja, a ventoinha seria desligada apenas quando a temperatura caísse abaixo de 29 °C. Isso evitaria que ela ficasse ligando e desligando repetidamente quando a temperatura oscila próxima ao limite.

4 NOMEAÇÃO E SINCRONIZAÇÃO

4.1 Nomeação

No contexto deste sistema distribuído, a nomeação serve para identificar de forma unívoca os recursos, os serviços e as origens de dados.

- **Identificação de tópicos (MQTT):** será utilizado um esquema de nomeação hierárquico, baseado em strings lógicas, para os tópicos MQTT. Exemplos: unoesc/grupo1/granja/temperatura, para o envio de dados, e unoesc/grupo1/granja/ventoinha, para o recebimento de comandos.
- **Identificação de componentes:** o Broker MQTT utilizado é o servidor público test.mosquitto.org, acessado pelo seu nome de domínio (resolvido via DNS) na porta 1883. O banco de dados MySQL é acessado pelo nome localhost, por executar na mesma máquina que o backend e o dashboard. As credenciais do banco são definidas de forma fixa no backend.

4.2 Sincronização

A comunicação via MQTT é assíncrona (modelo *Publish/Subscribe*), ou seja, o ESP32 e o backend não precisam estar em sincronia de execução no mesmo milissegundo para que o envio aconteça. No entanto, é necessária a sincronização de dados e de tempo:

- **Sincronização de relógios físicos:** na versão atual, o horário de cada leitura é registrado pelo banco de dados no momento da gravação feita pelo backend. Está previsto que o ESP32 faça requisições a um servidor NTP (*Network Time Protocol*) ao se conectar ao Wi-Fi, mantendo seu relógio sincronizado. Assim, cada leitura passaria a ser enviada com o *timestamp* (carimbo de tempo) real da ocorrência do dado, e não com o horário em que chegou ao banco.
- **Sincronização lógica:** as mensagens são trocadas com QoS 0 (*Quality of Service*), nível padrão do MQTT, que não confirma a entrega. Ainda assim, como o backend

envia um novo comando a cada leitura recebida, ou seja, a cada 5 segundos, um comando perdido ou recebido fora de ordem é corrigido automaticamente pelo comando seguinte, que sempre reflete a temperatura mais recente. A adoção do QoS 1, que garante a entrega ao menos uma vez, está prevista como melhoria.

5 COMPORTAMENTO EM CASO DE FALHAS DE COMUNICAÇÃO

O ambiente de uma granja costuma estar sujeito a quedas de internet e instabilidade de energia. As subseções a seguir descrevem como a versão atual do protótipo se comporta nessas situações e quais melhorias estão previstas para as próximas versões.

5.1 Tratamento dos dados durante a falha

Na versão atual, quando o ESP32 perde a conexão com o Broker MQTT, ele tenta se reconectar a cada 5 segundos e, enquanto isso, as leituras não são realizadas nem armazenadas. Como melhoria, prevê-se que o ESP32 continue lendo o sensor durante a falha e armazene as últimas leituras em um *buffer* (memória temporária) no próprio dispositivo. Também está prevista a atuação do ESP32 como *edge computing* (computação de borda): se a temperatura ultrapassar 30 °C durante a queda de rede, o próprio dispositivo acionaria a ventoinha localmente, como *fallback* de segurança, mesmo sem receber comandos do backend.

5.2 Informação apresentada ao usuário

Na versão atual, o dashboard exibe o horário da última leitura recebida, o que permite ao usuário perceber quando os dados deixam de ser atualizados. Caso o banco de dados esteja inacessível, é exibida uma mensagem de erro. Está previsto que, quando o intervalo desde a última atualização ultrapassar o limite (*threshold*) de 1 minuto, o que equivale a 12 leituras consecutivas não recebidas, o indicador de temperatura fique na cor cinza e seja exibido um alerta em vermelho com a mensagem **"Sensor Offline / Conexão Perdida"**.

5.3 Restabelecimento da comunicação

Quando a comunicação é restabelecida, o ESP32 se reconecta automaticamente ao Broker MQTT, assina novamente o tópico de comandos e retoma o envio das leituras, e o backend volta a exercer o controle da ventoinha. Com a implementação do *buffer*, prevê-se ainda que os dados acumulados durante a falha sejam enviados com seus respectivos *timestamps* originais, reconstruindo o histórico no banco de dados.

5.4 Identificação de dados desatualizados

Estão previstas duas formas de o sistema identificar que um dado está desatualizado:

1. **Tempo limite:** o dashboard verifica, pelo relógio do servidor, se a última inserção no banco ocorreu há mais tempo do que o limite estabelecido. Atualmente, essa verificação é feita visualmente pelo usuário, a partir do horário da última atualização exibido no dashboard.
2. **LWT** (*Last Will and Testament*): ao se conectar, o ESP32 informaria ao Broker que, caso se desconecte abruptamente, este deve publicar imediatamente uma mensagem de "offline" em um tópico específico. O backend leria essa mensagem e registraria a falha no banco de dados quase instantaneamente.

REFERÊNCIAS

AVIAGEN. **Ross frangos de corte**: manual de manejo. [S. l.]: Aviagen, 2018. Disponível em: https://pt.aviagen.com/assets/Tech_Center/BB_Foreign_Language_Docs/Portuguese/Ross-BroilerHandbook2018-PT.pdf. Acesso em: 23 ago. 2026.

CARVALHO, Fabyola Barros de. **Efeito da temperatura ambiente e da idade do frango de corte sobre o valor energético do milho, farelo e óleo de soja**. 2010. Dissertação (Mestrado em Zootecnia), Faculdade de Medicina Veterinária e Zootecnia, Universidade Estadual Paulista, Botucatu, 2010. Disponível em: https://www.fmvz.unesp.br/Home/ensino/pos-graduacao768/zootecnia/dissertacoeseteses/fabyola-barros-de-carvalho.pdf. Acesso em: 23 ago. 2026.